

Sorting III - Merge Sort and Quick Sort

Shahriar Ivan

Lecturer, Department of CSE, IUT



Merge Sort

The merge sort algorithm is defined recursively:

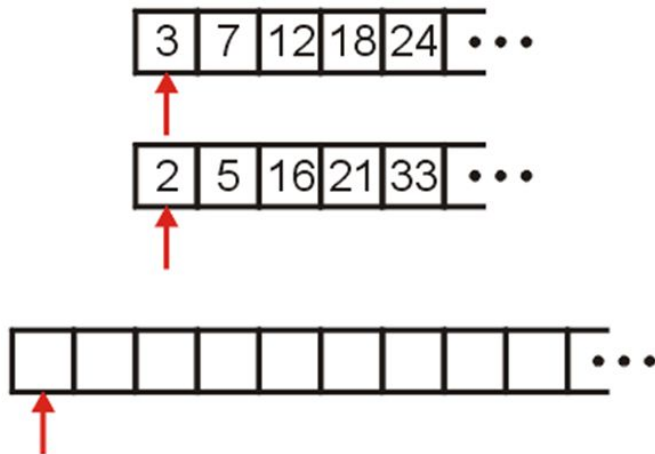
- If the list is of size 1, it is sorted—we are done;
- Otherwise:
 - Divide an unsorted list into two sub-lists,
 - Sort each sub-list recursively using merge sort, and
 - Merge the two sorted sub-lists into a single sorted list

This is the first significant divide-and-conquer algorithm we will see

Merging Example

Consider the two sorted arrays and an empty array

Define three indices at the start of each array

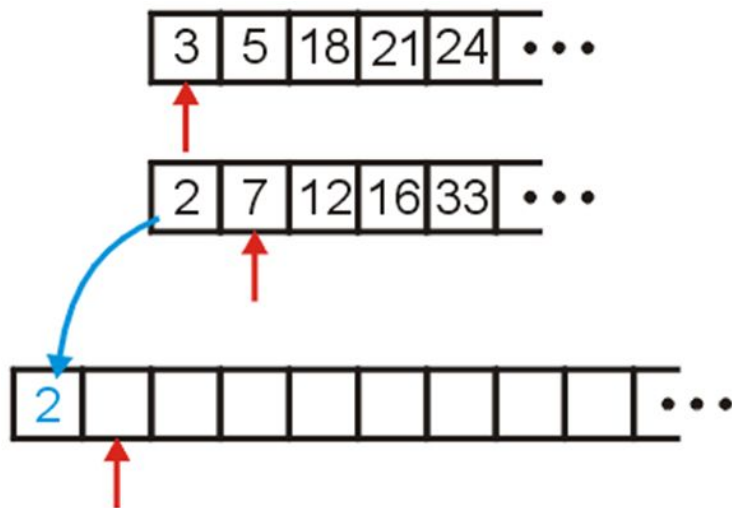


Merging Example

We compare 2 and 3: $2 < 3$

Copy 2 down

Increment the corresponding indices

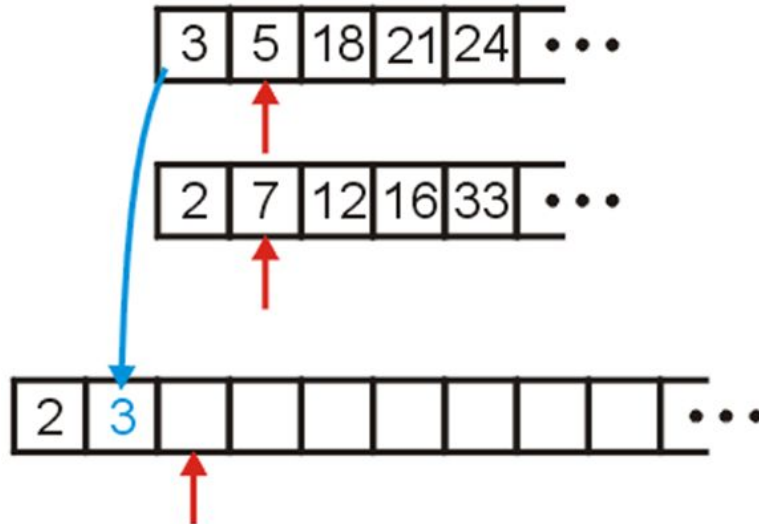


Merging Example

We compare 3 and 7

Copy 3 down

Increment the corresponding indices

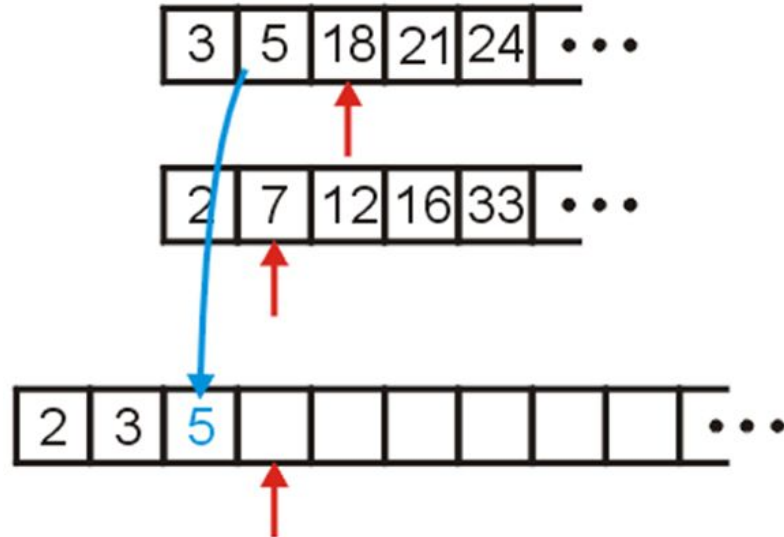


Merging Example

We compare 5 and 7

Copy 5 down

Increment the appropriate indices

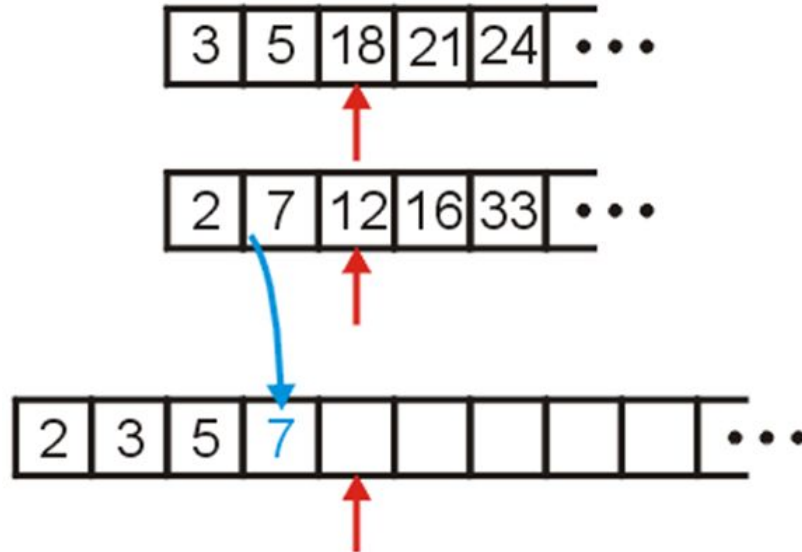


Merging Example

We compare 18 and 7

Copy 7 down

Increment...

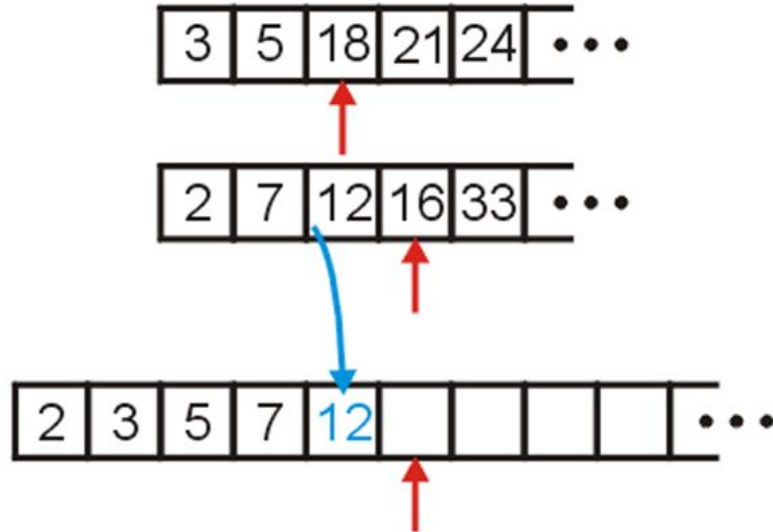


Merging Example

We compare 18 and 12

Copy 12 down

Increment...

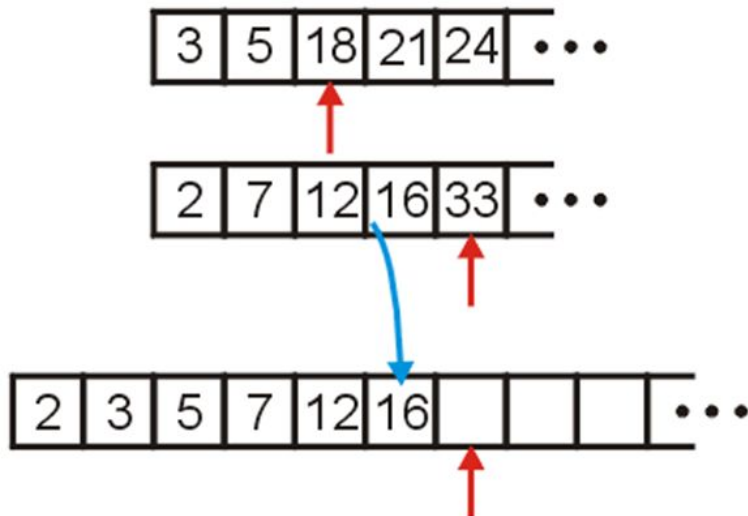


Merging Example

We compare 18 and 16

Copy 16 down

Increment...

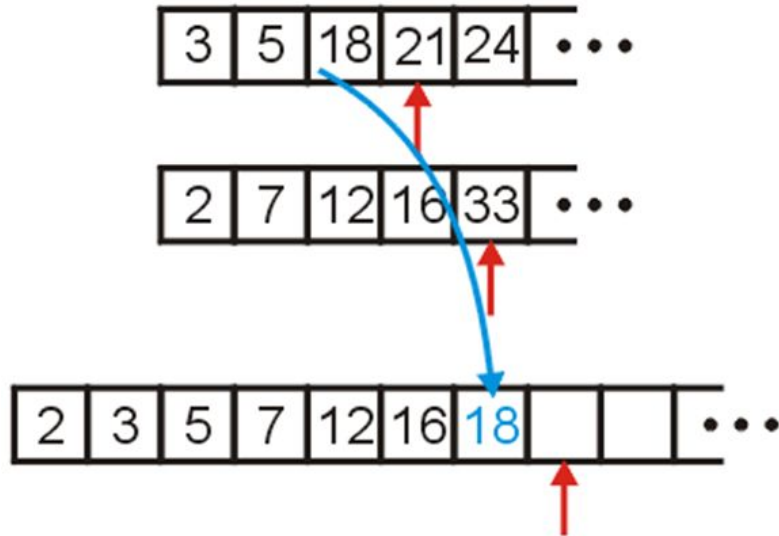


Merging Example

We compare 18 and 33

Copy 18 down

Increment...

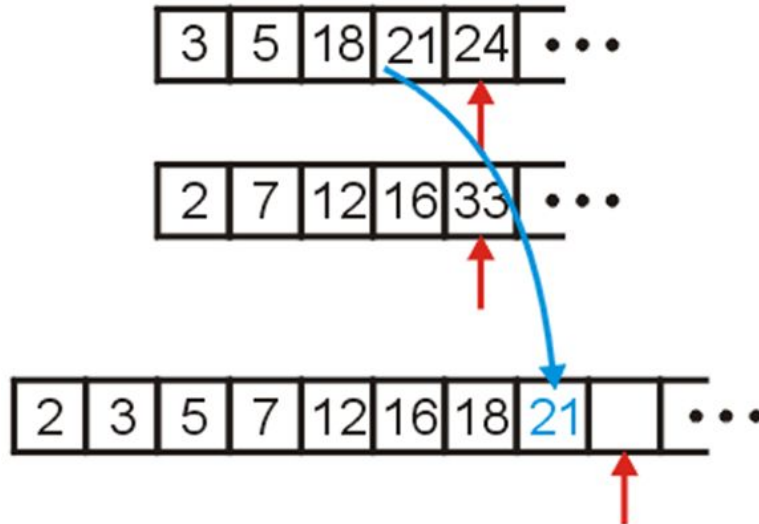


Merging Example

We compare 21 and 33

Copy 21 down

Increment...

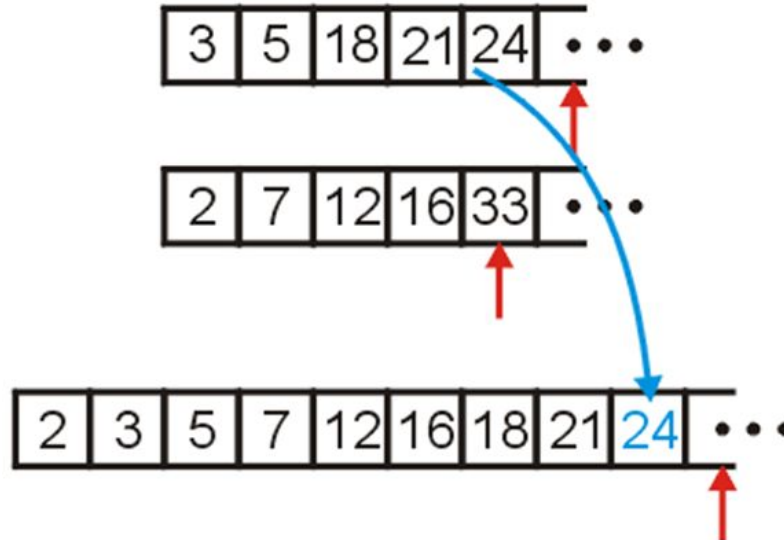


Merging Example

We compare 24 and 33

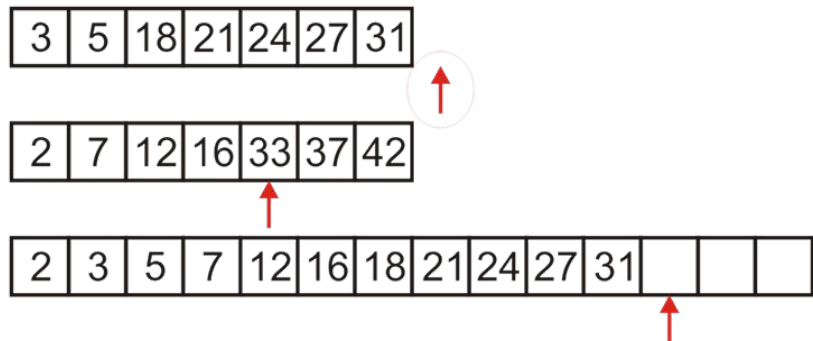
Copy 24 down

Increment...

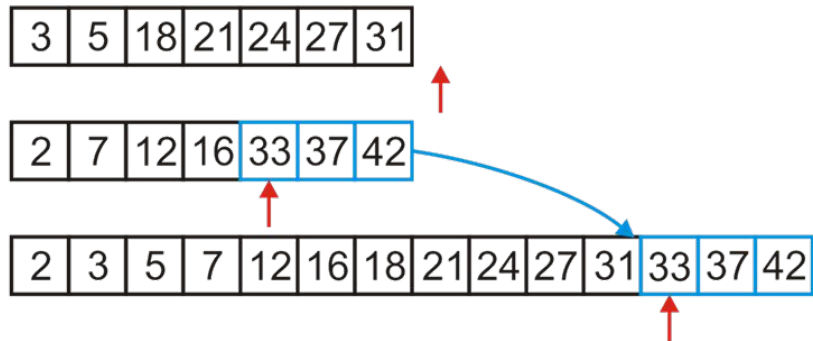


Merging Example

We would continue until we have passed beyond the limit of one of the two arrays



After this, we simply copy over all remaining entries in the non-empty array



Merging two lists

Programming a merge is straight-forward:

- the sorted arrays, array1 and array2, are of size n_1 and n_2 , respectively, and
- we have an empty array, arrayout, of size $n_1 + n_2$

Define three variables

```
int i1 = 0, i2 = 0, k = 0;
```

which index into these three arrays

Merging two lists

We can then run the following loop:

```
int i1 = 0, i2 = 0, k = 0;

while ( i1 < n1 && i2 < n2 ) {
    if ( array1[i1] < array2[i2] ) {
        arrayout[k] = array1[i1];
        ++i1;
    } else {
        assert( array1[i1] >= array2[i2] );
        arrayout[k] = array2[i2];
        ++i2;
    }
    ++k;
}
```

Merging two lists

We're not finished yet, we have to empty out the remaining array

```
for ( ; i1 < n1; ++i1, ++k ) {  
    arrayout[k] = array1[i1];  
}
```

```
for ( ; i2 < n2; ++i2, ++k ) {  
    arrayout[k] = array2[i2];  
}
```


Analysis of merging

The statement `++out` will only be run at most $n_1 + n_2$ times

Therefore, the body of the loops run a total of $n_1 + n_2$ times

Hence, merging may be performed in $\Theta(n_1 + n_2)$ time

If the arrays are approximately the same size, $n = n_1$ and $n_1 \approx n_2$, we can say that the run time is $\Theta(n)$

Problem: We cannot merge two arrays in-place

- This algorithm always required the allocation of a new array
- Therefore, the memory requirements are also $\Theta(n)$

The Algorithm

The algorithm:

- Split the list into two approximately equal sub-lists
- Recursively call merge sort on both sub lists
- Merge the resulting sorted lists

The Algorithm

Question:

- we split the list into two sub-lists and sorted them
- how should we sort those lists?

Answer (theoretical):

- if the size of these sub-lists is > 1 , use merge sort again
- if the sub-lists are of length 1, do nothing: a list of length one is sorted

The Algorithm

However, just because an algorithm has excellent asymptotic properties, this does not mean that it is practical at all levels

Answer (practical):

- If the sub-lists are less than some threshold length, use an algorithm like insertion sort to sort the lists
- Otherwise, use merge sort, again

Implementation

We will therefore implement a function

```
template <typename Type>
```

```
void merge_sort( Type *array, int first, int last );
```

that will sort the entries in the positions $\text{first} \leq i$ and $i < \text{last}$

- If the number of entries is less than N , call insertion sort
- Otherwise:
 - Find the mid-point,
 - Call merge sort recursively on each of the halves, and
 - Merge the results

Implementation

The actual body is quite small:

```
template <typename Type>
void merge_sort( Type *array, int first, int last ) {
    if ( last - first <= N ) {
        insertion_sort( array, first, last );
    } else {
        int midpoint = (first + last)/2;

        merge_sort( array, first, midpoint );
        merge_sort( array, midpoint, last );
        merge( array, first, midpoint, last );
    }
}
```

Quicksort

We have seen two $O(n \ln(n))$ sorting algorithms:

- Heap sort which allows in-place sorting, and
- Merge sort which is faster but requires more memory

We will now look at a recursive algorithm which may be done almost in place but which is faster than heap sort

- Use an object in the array (a pivot) to divide the two
- Average case: $\Theta(n \ln(n))$ time and $\Theta(\ln(n))$ memory
- Worst case: $\Theta(n^2)$ time and $\Theta(n)$ memory

We will look at strategies for avoiding the worst case

Quicksort

Merge sort splits the array sub-lists and sorts them

The larger problem is split into two sub-problems based on location in the array

Consider the following alternative:

- Chose an object in the array and partition the remaining objects into two groups relative to the chosen entry

Quicksort

For example, given

80	38	95	84	66	10	79	44	26	87	96	12	43	81	3
----	----	----	----	----	----	----	----	----	----	----	----	----	----	---

we can select the middle entry, 44, and sort the remaining entries into two groups, those less than 44 and those greater than 44:

38	10	26	12	43	3	44	80	95	84	66	79	87	96	81
----	----	----	----	----	---	----	----	----	----	----	----	----	----	----

Notice that 44 is now in the correct location if the list was sorted

Proceed by applying the algorithm to the first six and last eight entries

Run-time Analysis

Like merge sort, we can either:

- Apply insertion sort if the size of the sub-list is sufficiently small, or
- Sort the sub-lists using quicksort

In the best case, the list will be split into two approximately equal sub-lists, and thus, the run time could be very similar to that of merge sort: $\Theta(n \ln(n))$

What happens if we don't get that lucky?

Worst case scenario

Suppose we choose the first element as our pivot and we try ordering a sorted list:

80	38	95	84	66	10	79	2	26	87	96	12	43	81	3
----	----	----	----	----	----	----	---	----	----	----	----	----	----	---

Using 2, we partition into

2	80	38	95	84	66	10	79	26	87	96	12	43	81	3
---	----	----	----	----	----	----	----	----	----	----	----	----	----	---

We still have to sort a list of size $n - 1$

The run time is $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$

Thus, the run time drops from $n \ln(n)$ to n^2

Worst case scenario

Our goal is to choose the median element in the list as our pivot:

Unfortunately, it's difficult to find

Alternate strategy: take the median of a subset of entries

- For example, take the median of the first, middle, and last entries
- This will usually give a better approximation



Median of 3

Sorting the elements based on 44 results in two sub-lists, each of which must be sorted (again, using quicksort)

Select the 26 to partition the first sub-list:



Select 81 to partition the second sub-list:



Implementation

Note, however, we can do a better job with merge sort, it always divides the numbers being sorted into two equal or near-equal arrays

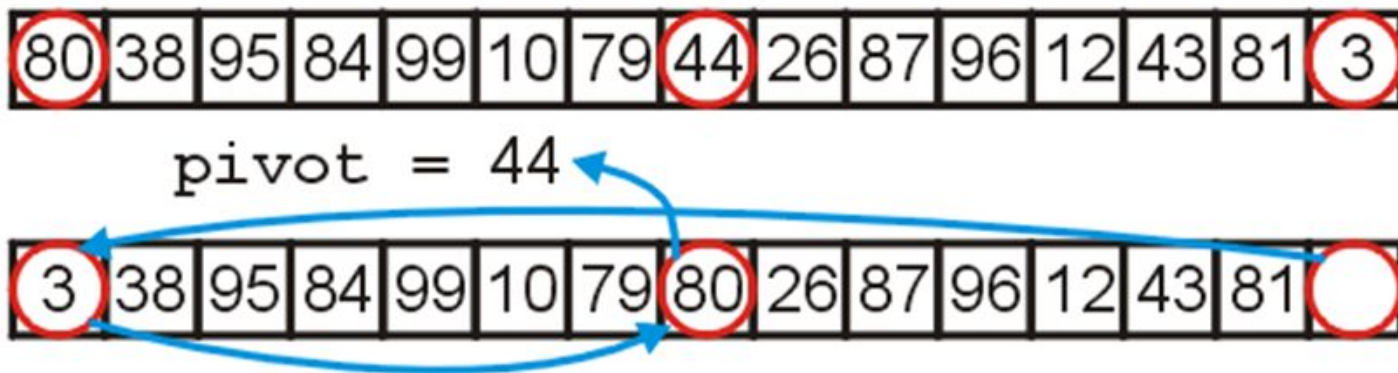
Can we implement quicksort in place?

Implementation

First, we have already examined the first, middle, and last entries and chosen the median of these to be the pivot

In addition, we can:

- move the smallest entry to the first entry
- move the largest entry to the middle entry



Implementation

Next, recall that our goal is to partition all remaining elements based on whether they are smaller than or greater than the pivot

We will find two entries:

- One larger than the pivot (starting from the front)
- One smaller than the pivot (starting from the back)

which are out of order and then swap them

Implementation

- Continue doing so until the appropriate entries you find are actually in order
- The index to the larger entry we found would be the first large entry in the list (as seen from the left)
- Therefore, we could move this entry into the last entry of the list
- We can fill this spot with the pivot

Quicksort Example

Consider the following unsorted array of 25 entries

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	77	49	35	61	48	73	23	95	3	89	37	57	99	17	32	94	28	15	55	7	51	88	97	62

We will call insertion sort if the list being sorted of size $N = 6$ or less

We call quicksort(array, 0, 25)

Quicksort Example

We are calling quicksort(array, 0, 25)

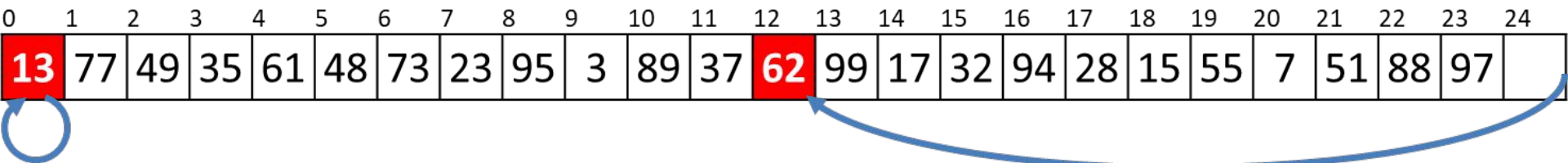
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	77	49	35	61	48	73	23	95	3	89	37	57	99	17	32	94	28	15	55	7	51	88	97	62

First, $25 - 0 > 6$, so find the midpoint and the pivot

$\text{midpoint} = (0 + 25)/2; // == 12$

Quicksort Example

We are calling quicksort(array, 0, 25)



First, $25 - 0 > 6$, so find the midpoint and the pivot

$\text{midpoint} = (0 + 25) / 2; // == 12$

$\text{pivot} = 57;$

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	77	49	35	61	48	73	23	95	3	89	37	62	99	17	32	94	28	15	55	7	51	88	97	

Starting from the front and back:

Find the next element greater than the pivot

The last element less than the pivot

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	77	49	35	61	48	73	23	95	3	89	37	62	99	17	32	94	28	15	55	7	51	88	97	

Searching forward and backward:

low = 1;

high = 21;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	61	48	73	23	95	3	89	37	62	99	17	32	94	28	15	55	7	77	88	97	



Searching forward and backward:

low = 1;

high = 21;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	61	48	73	23	95	3	89	37	62	99	17	32	94	28	15	55	7	77	88	97	

Continue searching

low = 4;

high = 20;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	73	23	95	3	89	37	62	99	17	32	94	28	15	55	61	77	88	97	



Continue searching

low = 4;

high = 20;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	73	23	95	3	89	37	62	99	17	32	94	28	15	55	61	77	88	97	

Continue searching

low = 6;

high = 19;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	95	3	89	37	62	99	17	32	94	28	15	73	61	77	88	97	



Continue searching

low = 6;

high = 19;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	95	3	89	37	62	99	17	32	94	28	15	73	61	77	88	97	

Continue searching

low = 8;

high = 18;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	89	37	62	99	17	32	94	28	95	73	61	77	88	97	



Continue searching

low = 8;

high = 18;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	89	37	62	99	17	32	94	28	95	73	61	77	88	97	

Continue searching

low = 10;

high = 17;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	62	99	17	32	94	89	95	73	61	77	88	97	



Continue searching

low = 10;

high = 17;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	62	99	17	32	94	89	95	73	61	77	88	97	

Continue searching

low = 12;

high = 15;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	32	99	17	62	94	89	95	73	61	77	88	97	



Continue searching

low = 12;

high = 15;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	32	99	17	62	94	89	95	73	61	77	88	97	

Continue searching

low = 13;

high = 14;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	32	17	99	62	94	89	95	73	61	77	88	97	



Continue searching

low = 13;

high = 14;

Swap them

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	32	17	99	62	94	89	95	73	61	77	88	97	

Continue searching

low = 14;

high = 13;

Now, low > high, so we stop

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	32	17	57	62	94	89	95	73	61	77	88	97	99

Continue searching

low = 14;

high = 13;

Now, low > high, so we stop

pivot = 57;

Quicksort Example

We are calling quicksort(array, 0, 25)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
13	51	49	35	7	48	55	23	15	3	28	37	32	17	57	62	94	89	95	73	61	77	88	97	99

We now begin calling quicksort recursively on the other two halves

```
quicksort( array, 0, 14 );
```

```
quicksort( array, 15, 25 );
```

pivot = 57;

Overall Implementation

```
template <typename Type>
void quicksort( Type *array, int first, int last ) {
    if ( last - first <= N ) {
        insertion_sort( array, first, last );
    } else {
        Type pivot = find_pivot( array, first, last );
        int low  = find_next( pivot, array, first + 1 );
        int high = find_previous( pivot, array, last - 2 );

        while ( low < high ) {
            std::swap( array[low], array[high] );
            low  = find_next( pivot, array, low + 1 );
            high = find_previous( pivot, array, high - 1 );
        }

        array[last - 1] = array[low];
        array[low] = pivot;
        quicksort( array, first, low );
        quicksort( array, high, last );
    }
}
```

End of Lecture